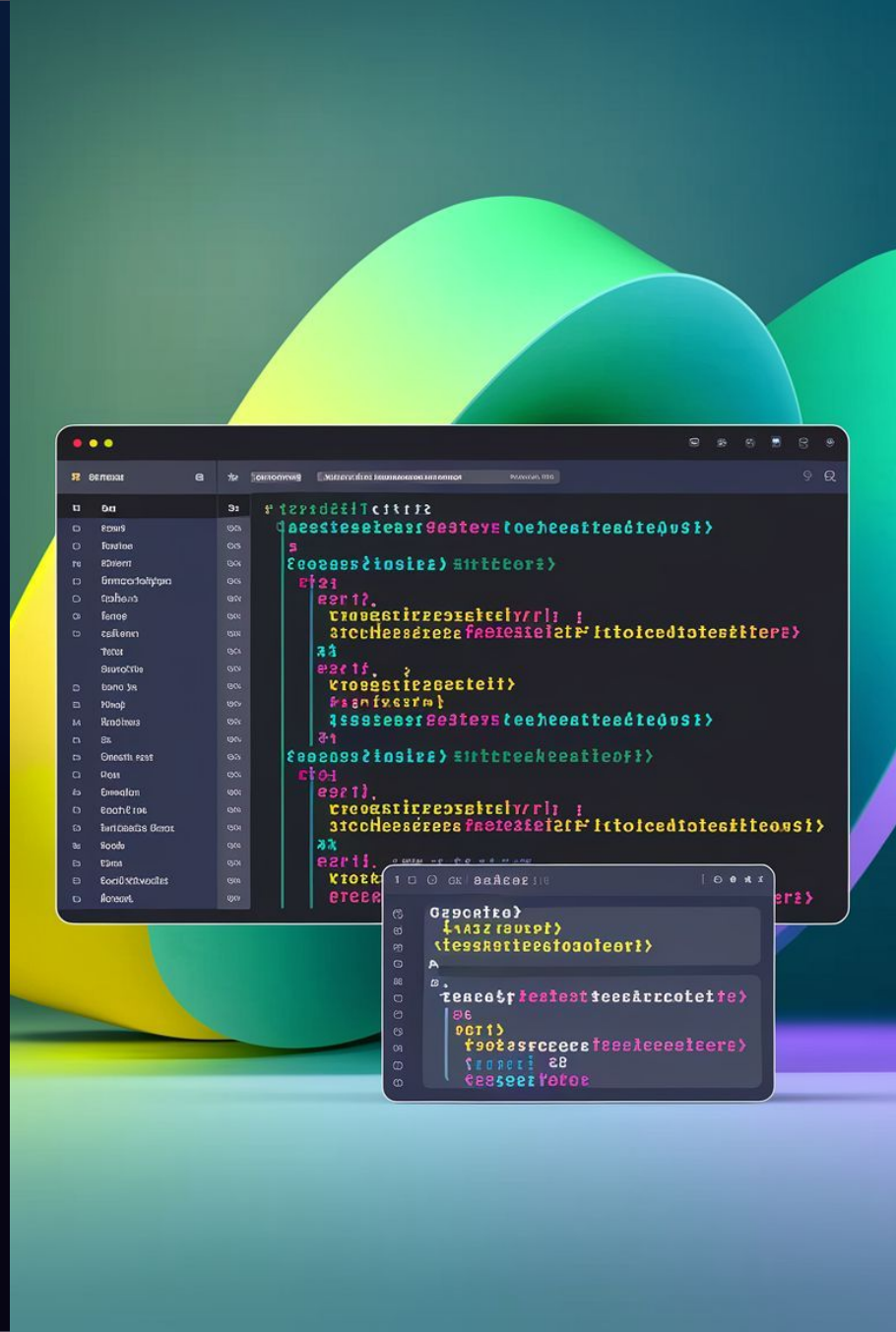
JavaScript für Anfänger: Ein umfassender Überblick

Entdecken Sie die Grundlagen von JavaScript und wie Sie diese vielseitige Sprache für interaktive Webseiten nutzen können.



getElementById() und querySelector(): Elemente gezielt ansprechen

getElementById()

Diese Funktion ermöglicht es, ein spezifisches HTML-Element über seine eindeutige ID anzusprechen. Dies ist besonders nützlich, wenn Sie ein Element direkt manipulieren möchten, beispielsweise um Text zu ändern oder Stile anzupassen.

```
document.getElementById("demo").innerText = "Hallo!";
```

Ein Anwendungsbeispiel wäre das Ändern des Inhalts eines Div-Elements mit der ID "demo" zu "Hallo Welt!".

querySelector()

querySelector() hingegen bietet mehr Flexibilität, da es Elemente anhand von CSS-Selektoren auswählt. So können Sie Elemente nach Klasse, Attribut oder Tag ansprechen.

```
document.querySelector(".demo-class").style.color = "blue";
```

Beispielsweise können Sie alle Elemente mit der Klasse "demo-class" auswählen und deren Textfarbe auf Blau setzen.

setTimeout(), fetch() und Array.forEach(): Zeit, Daten und Iteration



setTimeout()

Mit setTimeout() können Sie die Ausführung von Code verzögern. Dies ist ideal für Animationen, das Anzeigen von Nachrichten nach einer bestimmten Zeit oder das regelmäßige Ausführen von Funktionen.

```
setTimeout(function() {
alert("Zeit abgelaufen!"); },
3000);
```



fetch()

fetch() ist die moderne Art, Daten von einem Server abzurufen. Es unterstützt Promises und ermöglicht das einfache Abrufen von JSON-Daten oder anderen Ressourcen.

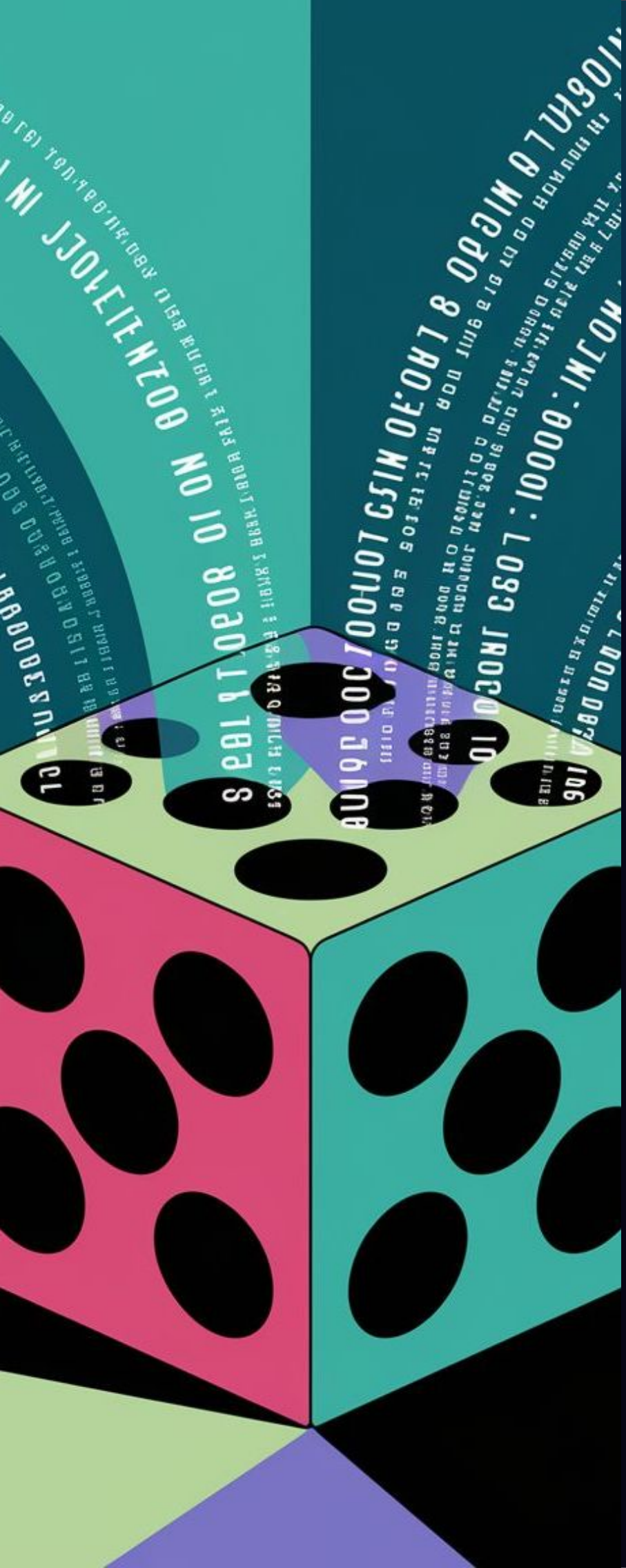
```
fetch('https://api.example.co
m/data').then(response =>
response.json()).then(data =>
console.log(data));
```



Array.forEach()

Array.forEach() ist eine praktische Methode, um jedes Element in einem Array zu durchlaufen und eine Funktion darauf anzuwenden. Dies ist nützlich für das Anzeigen von Daten oder das Durchführen von Berechnungen.

```
[1, 2, 3].forEach(num =>
console.log(num));
```



JSON.parse(), JSON.stringify() und Math.random(): Datenwandlung und Zufall

1 JSON.parse()

Diese Funktion wandelt einen JSON-String in ein JavaScript-Objekt um. Dies ist entscheidend, um Daten zu verarbeiten, die von APIs empfangen werden.

```
let obj = JSON.parse('{"name":"Max"}');
```

2 JSON.stringify()

Das Gegenteil von JSON.parse(): Es konvertiert ein JavaScript-Objekt in einen JSON-String. Dies ist wichtig, um Daten an einen Server zu senden.

```
let jsonStr = JSON.stringify({name: "Max"});
```

3 Math.random()

Math.random() erzeugt eine Zufallszahl zwischen 0 und 1. Dies ist nützlich für Spiele, Simulationen oder zufällige Auswahlen.

```
let zufallszahl = Math.random();
```

Array.map(): Transformation von Array-Daten

Anwendungsbereich

Array.map() ist ideal, um jedes Element eines Arrays zu transformieren und ein neues Array mit den Ergebnissen zu erstellen. Dies ist besonders nützlich, wenn Sie Daten umwandeln oder formatieren müssen.

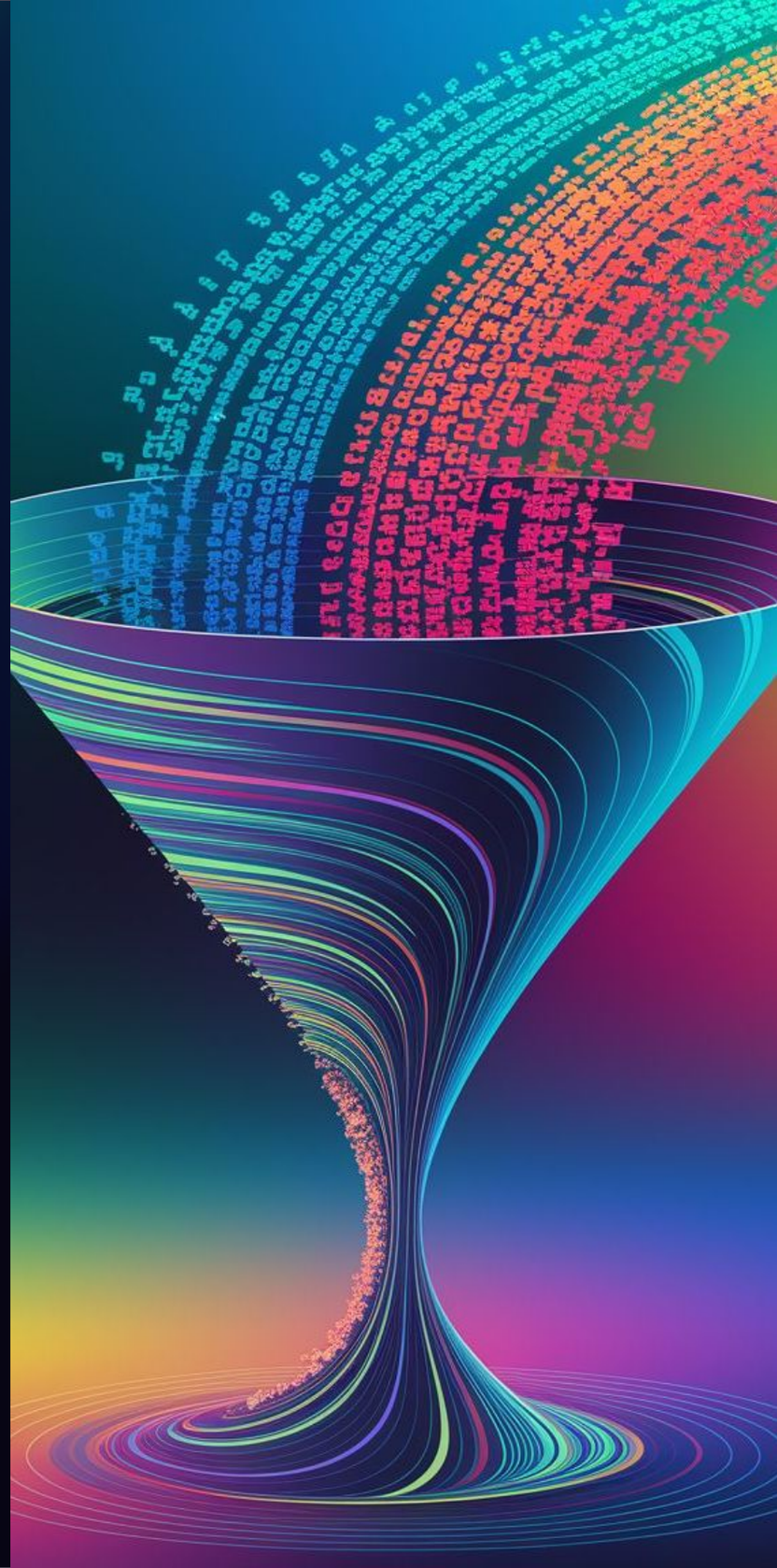
Beispiel

Nehmen wir an, Sie haben ein Array von Zahlen und möchten jedes Element verdoppeln. Mit Array.map() ist das einfach:

```
[1, 2, 3].map(num => num * 2); // Ergebnis: [2, 4, 6]
```

Vorteile

Array.map() ist übersichtlich, effizient und vermeidet Seiteneffekte, da es ein neues Array erzeugt, ohne das ursprüngliche Array zu verändern.



Array.filter(), Array.reduce(), String.replace() und Date.now(): Daten filtern, reduzieren, ersetzen und Zeit erfassen

1 2 3 4

Array.filter()

Erzeugt ein neues Array mit allen Elementen, die einen bestimmten Test bestehen.

```
[1, 2, 3].filter(num => num > 1); // [2, 3]
```

Array.reduce()

Reduziert ein Array auf einen einzigen Wert durch Anwendung einer Funktion auf jedes Element.

```
[1, 2, 3].reduce((acc, num) => acc + num, 0); // 6
```

String.replace()

Ersetzt einen bestimmten Wert in einem String durch einen anderen Wert.

```
'Hello World'.replace('World', 'AI'); // Hello AI
```

Date.now()

Gibt die Anzahl der Millisekunden zurück, die seit dem 1. Januar 1970 vergangen sind.

```
let now = Date.now();
```

Object.keys(), Object.values() und Object.assign(): Objekt-Eigenschaften extrahieren und zusammenführen

Object.keys()

Gibt ein Array mit den Namen der aufzählbaren Eigenschaften eines Objekts zurück.

```
let keys = Object.keys({a: 1, b: 2}); // ['a', 'b']
```



Object.values()

Gibt ein Array mit den Werten der aufzählbaren Eigenschaften eines Objekts zurück.

```
let values = Object.values({a: 1, b: 2}); // [1, 2]
```

Object.assign()

Kopiert die Werte aller aufzählbaren Eigenschaften von einem oder mehreren Quellobjekten in ein Zielobjekt.

```
let target = {};
Object.assign(target, {a: 1}, {b: 2}); // {a: 1, b: 2}
```

Math.floor() und Math.ceil(): Zahlen runden

Math.floor()

Rundet eine Zahl abwärts zur nächsten ganzen Zahl.

```
let rounded = Math.floor(4.7); // 4
```

Math.ceil()

Rundet eine Zahl aufwärts zur nächsten ganzen Zahl.

```
let rounded = Math.ceil(4.3); // 5
```



JSON.parse(): JSON-Strings in JavaScript-Objekte umwandeln

Anwendungsbereich

JSON.parse() ist entscheidend, um Daten zu verarbeiten, die im JSON-Format vorliegen. Dies ist typisch, wenn Sie Daten von Web-APIs empfangen.

Beispiel

Angenommen, Sie erhalten einen JSON-String von einem Server. Mit JSON.parse() können Sie diesen in ein JavaScript-Objekt umwandeln:

```
let obj =
JSON.parse('{"name":
"John"}');
```

Anschließend können Sie auf die Eigenschaften des Objekts zugreifen, z.B. obj.name, um den Wert "John" zu erhalten.

Wichtiger Hinweis

Stellen Sie sicher, dass der JSON-String korrekt formatiert ist, da JSON.parse() sonst einen Fehler ausgibt.

Array.some() und Array.every(): Bedingungen in Arrays prüfen

1

Array.some()

Prüft, ob mindestens ein Element in einem Array einen bestimmten Test besteht.

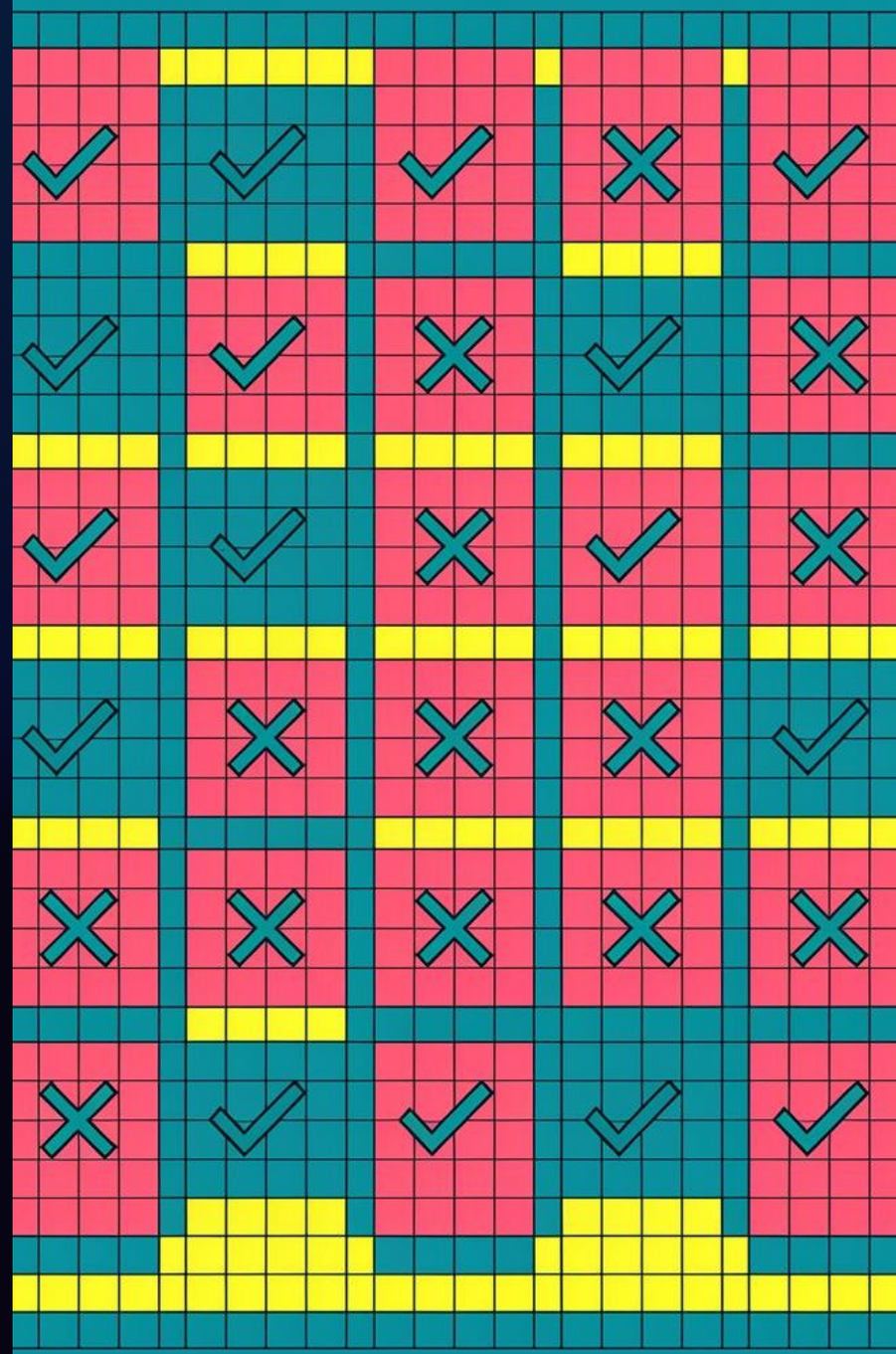
```
[1, 2, 3].some(num => num > 2); // true
```

2

Array.every()

Prüft, ob alle Elemente in einem Array einen bestimmten Test bestehen.

```
[1, 2, 3].every(num => num < 5); // true
```



String.indexOf() und String.slice(): Strings durchsuchen und extrahieren



String.indexOf()

Findet die Position eines bestimmten Worts in einem String.

```
'Hello World'.indexOf('World'); // 6
```



String.slice()

Extrahiert einen Teil eines Strings ab einer bestimmten Position.

```
'Hello World'.slice(6); // ' World'
```

String.toUpperCase() und String.toLowerCase(): Groß- und Kleinschreibung ändern

1 **String.toUpperCase()**

Wandelt einen String in Großbuchstaben um.

```
'hello'.toUpperCase(); // 'HELLO'
```

2 **String.toLowerCase()**

Wandelt einen String in Kleinbuchstaben um.

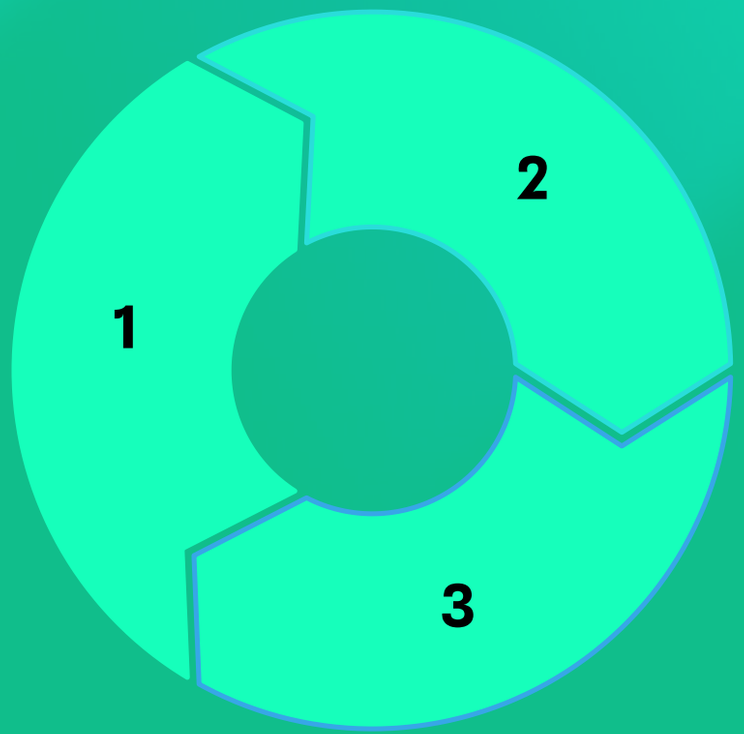
```
'HELLO'.toLowerCase(); // 'hello'
```

Number.isNaN(), Number.isInteger() und Number.parseFloat(): Zahlen prüfen und umwandeln

Number.isNaN()

Prüft, ob ein Wert NaN (Not a Number) ist.

```
Number.isNaN(NaN); // true
```



Number.isInteger()

Prüft, ob ein Wert eine ganze Zahl ist.

```
Number.isInteger(4); // true
```

Number.parseFloat()

Wandelt einen String in eine Gleitkommazahl um.

```
let num = Number.parseFloat('4.7'); // 4.7
```

Number.parseInt(), Array.some() und Array.every(): Zahlen prüfen und Bedingungen in Arrays

1

Number.parseInt()

Wandelt einen String in eine ganze Zahl um.

```
let num = Number.parseInt('4'); // 4
```

2

Array.some()

Prüft, ob mindestens ein Element in einem Array einen bestimmten Test besteht.

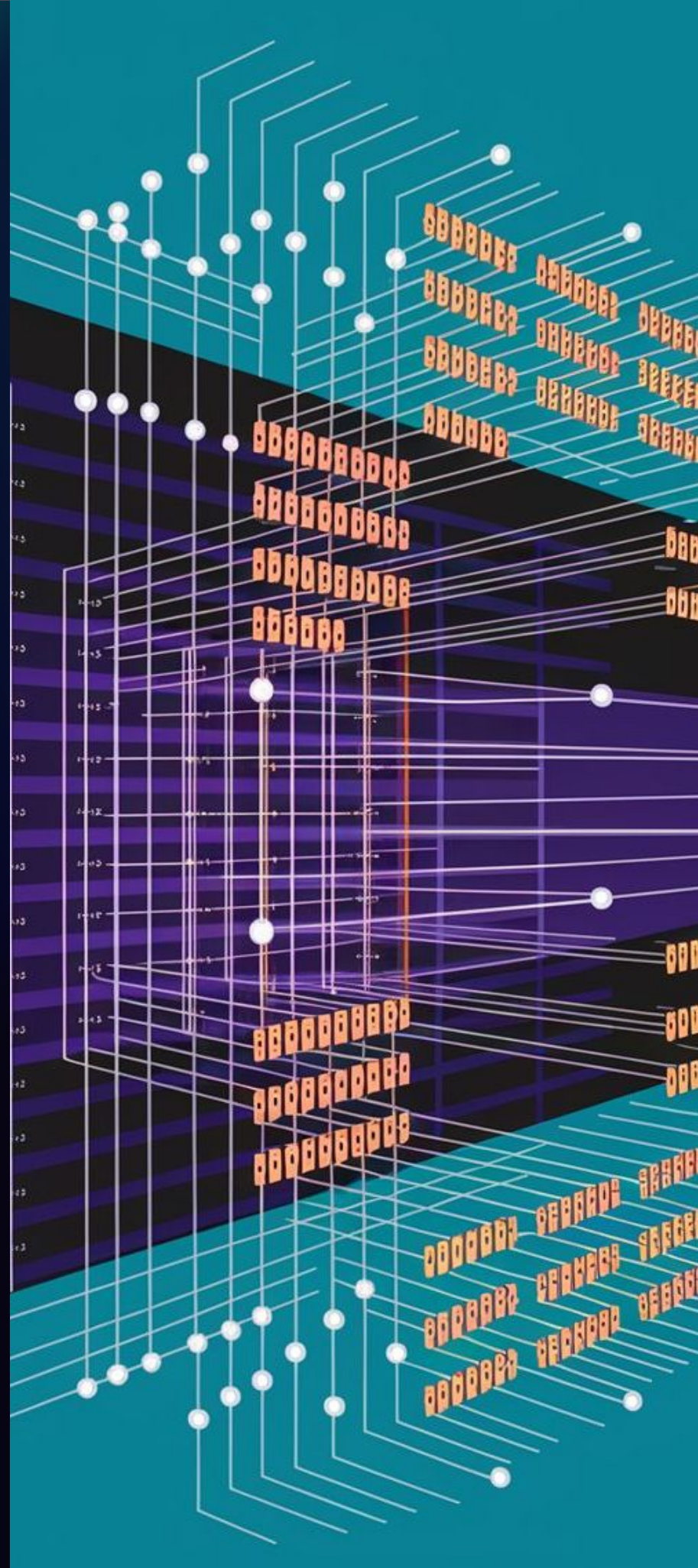
```
[1, 2, 3].some(num => num > 2); // true
```

3

Array.every()

Prüft, ob alle Elemente in einem Array einen bestimmten Test bestehen.

```
[1, 2, 3].every(num => num < 5); // true
```



String.indexOf(), String.slice() und String.toUpperCase(): Strings manipulieren

String.indexOf()

Findet die Position eines bestimmten Worts in einem String.

```
'Hello World'.indexOf('World'); // 6
```

String.slice()

Extrahiert einen Teil eines Strings ab einer bestimmten Position.

```
'Hello World'.slice(6); // 'World'
```

String.toUpperCase()

Wandelt einen String in Großbuchstaben um.

```
'hello'.toUpperCase(); // 'HELLO'
```

String.toLowerCase(), Number.isNaN() und Number.isInteger(): Strings und Zahlen prüfen

1

2

3

String.toLowerCase()

Wandelt einen String in Kleinbuchstaben um.

```
'HELLO'.toLowerCase(); // 'hello'
```

Number.isNaN()

Prüft, ob ein Wert NaN (Not a Number) ist.

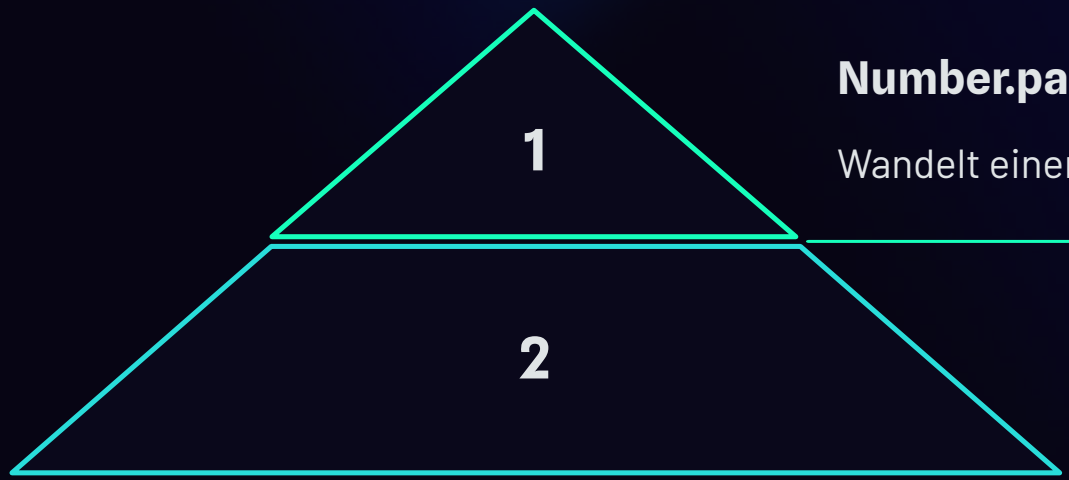
```
Number.isNaN(NaN); // true
```

Number.isInteger()

Prüft, ob ein Wert eine ganze Zahl ist.

```
Number.isInteger(4); // true
```

Number.parseFloat() und Number.parseInt(): Strings in Zahlen umwandeln



Number.parseFloat()

Wandelt einen String in eine Gleitkommazahl um.

Number.parseInt()

Wandelt einen String in eine ganze Zahl um.

Promise() und async/await: Asynchrone Operationen handhaben

Promise()

Ein Objekt, das den eventuellen Abschluss (oder das Fehlschlagen) einer asynchronen Operation repräsentiert.

```
let promise = new Promise((resolve, reject) => {
/* asynchrone Operation */ });
```

async/await

Eine Syntax, um mit Promises in einer komfortableren, synchronen Art und Weise zu arbeiten.

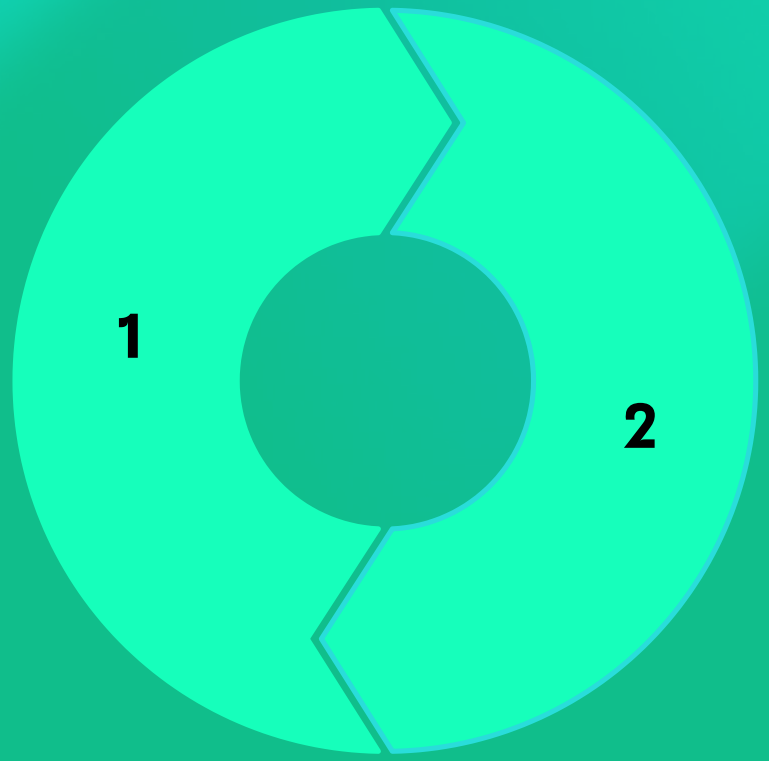
```
async function example() { let result = await
promise; }
```

Object.create() und Object.defineProperty(): Objekte erstellen und Eigenschaften definieren

Object.create()

Erstellt ein neues Objekt mit dem angegebenen Prototypobjekt und Eigenschaften.

```
let obj = Object.create({}, {
  prop: { value: 'value',
  writable: true,
  enumerable: true,
  configurable: true } });
```



Object.defineProperty()

Definiert eine neue Eigenschaft direkt auf einem Objekt oder ändert eine bestehende Eigenschaft auf einem Objekt.

```
Object.defineProperty(obj,
'prop', { value: 'value',
writable: true,
enumerable: true,
configurable: true });
```

Object.freeze(): Objekte unveränderlich machen

Anwendungsbereich

Object.freeze() friert ein Objekt ein und verhindert jegliche Änderungen an seinen Eigenschaften. Dies ist nützlich, um sicherzustellen, dass ein Objekt nicht versehentlich verändert wird.

Beispiel

```
let obj = { prop: 'value' }; Object.freeze(obj);
```

Nach dem Einfrieren kann keine Eigenschaft des Objekts mehr geändert oder hinzugefügt werden.



Array.from() und Symbol(): Arrays erstellen und eindeutige Eigenschaften definieren

1

2

Array.from()

Erstellt eine neue Array-Instanz aus einem array-ähnlichen oder iterierbaren Objekt.

```
let arr = Array.from('Hello'); // ['H', 'e', 'l', 'l', 'o']
```

Symbol()

Erstellt einen eindeutigen und unveränderlichen primitiven Wert, der als Schlüssel einer Objekteigenschaft verwendet werden kann.

```
let sym = Symbol('description'); let obj = {
[sym]: 'value' };
```

Set: Eindeutige Werte speichern



Anwendungsbereich

Ein Set ist eine Sammlung eindeutiger Werte. Dies ist nützlich, um Duplikate zu entfernen oder sicherzustellen, dass nur eindeutige Werte gespeichert werden.

Beispiel

```
let set = new Set([1, 2, 3, 4]);
```

Wenn Sie versuchen, einen doppelten Wert hinzuzufügen, wird dieser ignoriert.

Map: Schlüssel-Wert-Paare speichern

1

Anwendungsbereich

Eine Map speichert Schlüssel-Wert-Paare und erinnert sich an die ursprüngliche Einfügereihenfolge der Schlüssel.

2

Beispiel

```
let map = new Map([['key', 'value'], ['key2', 'value2']]);
```

Sie können jeden Datentyp als Schlüssel verwenden.

Proxy: Objekte abfangen und anpassen

Anwendungsbereich

Ein Proxy ermöglicht es, Operationen auf einem Objekt abzufangen und anzupassen. Dies ist nützlich für Validierung, Protokollierung oder andere erweiterte Szenarien.

Beispiel

```
let handler = { get: function(target, name) { /* benutzerdefinierte Logik */ } }; let proxy = new Proxy({}, handler);
```

Sie können beispielsweise den Zugriff auf Eigenschaften protokollieren oder Validierungen durchführen.



Reflect: JavaScript-Operationen abfangen

1

Anwendungsbereich

Reflect bietet Methoden, um abfangbare JavaScript-Operationen auszuführen, die von Proxy-Handlern verwendet werden.

```
Reflect.get(obj, 'prop');
```

2

Beispiel

Sie können Reflect verwenden, um Eigenschaften abzurufen, zu setzen oder zu löschen.



Generatoren: Ausführung von Funktionen steuern

1 Anwendungsbereich

Generatoren sind spezielle Funktionen, die es ermöglichen, ihre Ausführung zu steuern, indem sie iteriert werden. Sie verwenden das Schlüsselwort yield, um die Ausführung zu pausieren und fortzusetzen.

2 Beispiel

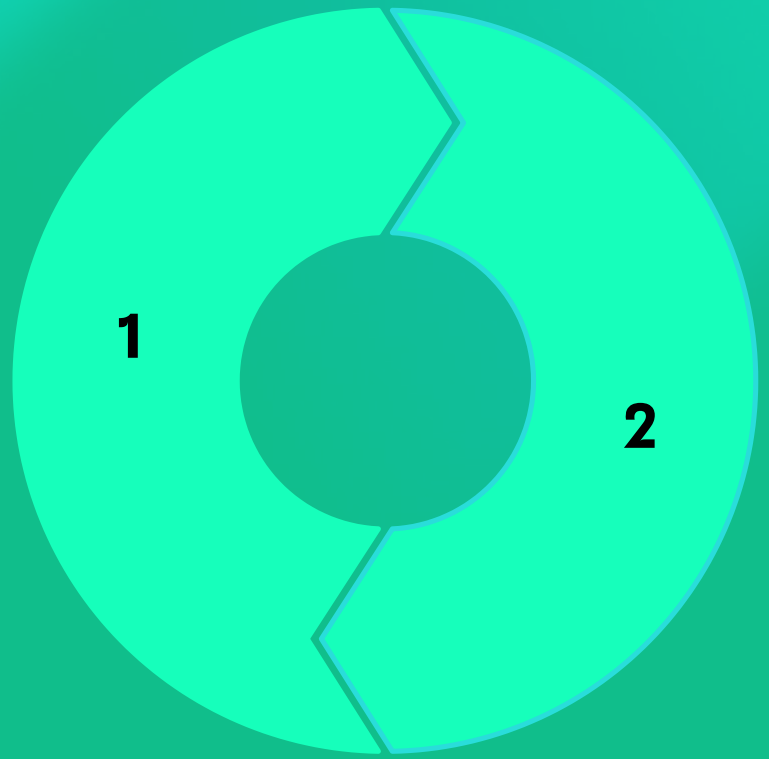
```
function* generator() { yield 1; yield 2; yield 3; }
```

Sie können einen Generator verwenden, um benutzerdefinierte iterierbare Objekte zu erstellen.

Async Generators: Asynchrone iterierbare Objekte erstellen

Anwendungsbereich

Async Generators kombinieren asynchrone Funktionen und Generatoren, um asynchrone iterierbare Objekte zu erstellen.



Beispiel

```
async function*
asyncGenerator() { yield
await promise; }
```

Dies ermöglicht es, asynchrone Operationen in einer iterierbaren Sequenz zu verarbeiten.

WeakSet: Sammlungen von schwach gehaltenen Objekten

1

Anwendungsbereich

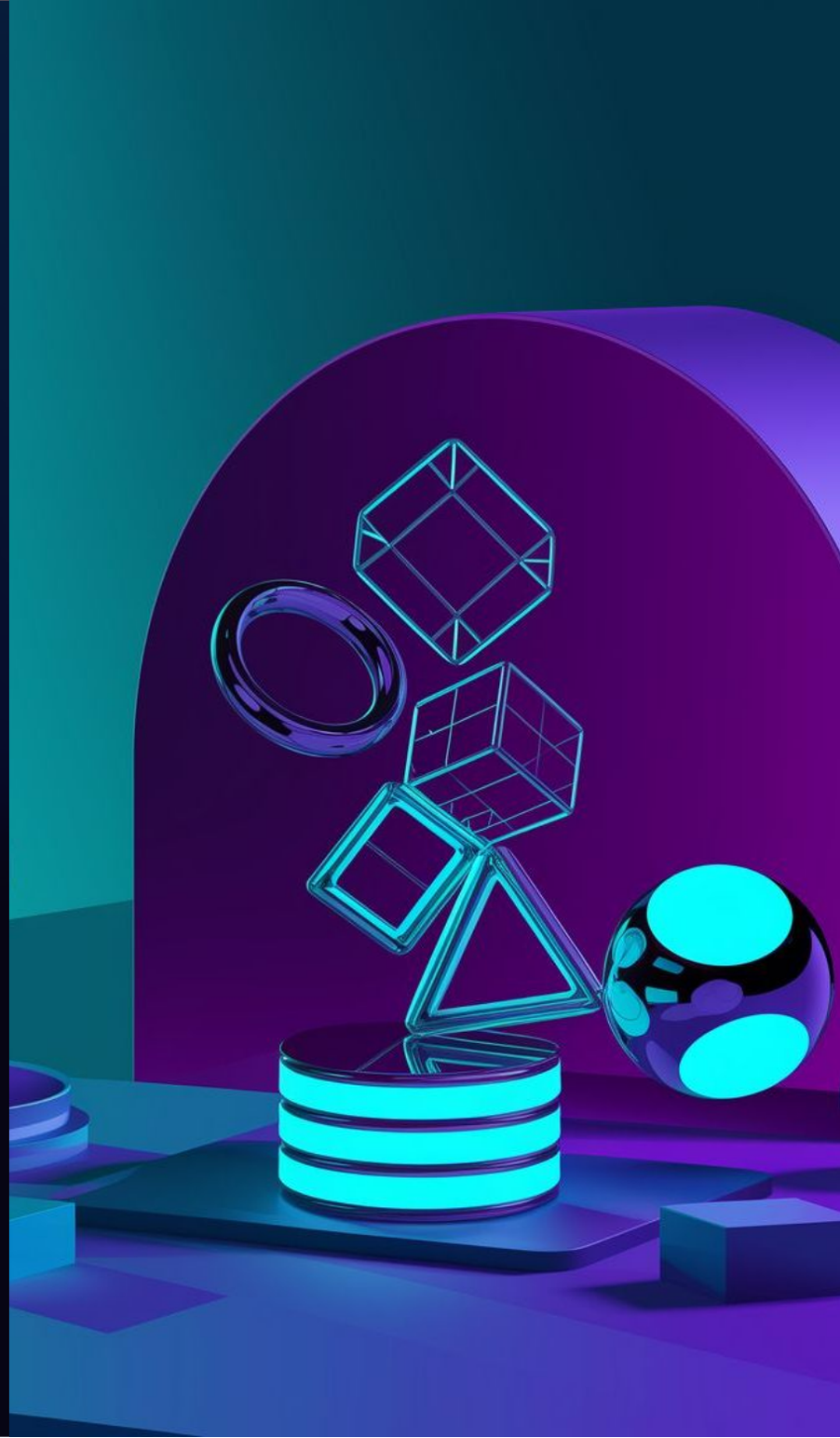
Ein WeakSet ist eine Sammlung eindeutiger Objekte, wobei die Objekte schwach gehalten werden. Dies bedeutet, dass sie automatisch vom Garbage Collector entfernt werden können, wenn keine anderen Referenzen auf sie vorhanden sind.

2

Beispiel

```
let weakSet = new WeakSet(); weakSet.add({ key: 'value' });
```

Dies ist nützlich, um Speicherlecks zu vermeiden.





WeakMap: Sammlungen von schwach gehaltenen Schlüssel-Wert-Paaren

Anwendungsbereich

Ein WeakMap ist eine Sammlung von Schlüssel-Wert-Paaren, wobei die Schlüssel schwach gehalten werden. Dies bedeutet, dass die Schlüssel automatisch vom Garbage Collector entfernt werden können, wenn keine anderen Referenzen auf sie vorhanden sind.

Beispiel

```
let weakMap = new WeakMap(); weakMap.set({ key: 'value' }, 'value2');
```

Dies ist nützlich, um Speicherlecks zu vermeiden und Metadaten zu Objekten zu speichern, ohne deren Lebenszyklus zu beeinflussen.

Class: Blaupausen für Objekte erstellen

1

2

Anwendungsbereich

Eine Klasse ist eine Syntax zur Erstellung von Objekten mit einem Konstruktor und Prototypmethoden. Sie dient als Blaupause für die Erstellung neuer Objekte.

Beispiel

```
class MyClass { constructor() { /*...*/ }
method() { /*...*/ } }
```

Klassen erleichtern die objektorientierte Programmierung.

Module: Code organisieren und wiederverwenden



Anwendungsbereich

Ein Modul ist eine Datei, die JavaScript-Code enthält, der importiert und exportiert werden kann. Dies ermöglicht es, Code zu organisieren und in verschiedenen Dateien wiederzuverwenden.



Beispiel

```
export function myFunction() { /*...*/ }
```

```
import { myFunction } from './module.js';
```

Module verbessern die Code-Wartbarkeit und Wiederverwendbarkeit.

Template Literals: Dynamische Strings erstellen

Anwendungsbereich

Template Literals sind eine Syntax zur Erstellung von Strings mit eingebetteten Ausdrücken. Sie verwenden Backticks (\`) und ${expression} Platzhalter.

Beispiel

```
let str = `Hallo, ${name}!`;
```

Template Literals erleichtern die Erstellung dynamischer Strings.

Destructuring Assignment: Werte aus Arrays und Objekten extrahieren

Anwendungsbereich

Destructuring Assignment ist eine Syntax zur Extraktion von Werten aus Arrays oder Objekten und deren Zuweisung zu Variablen.



Beispiel

```
let [a, b] = [1, 2];
```

```
let { prop } = { prop: 'value' };
```

Dies vereinfacht den Umgang mit komplexen Datenstrukturen.

Spread Operator: Elemente verteilen und Objekte zusammenführen

Anwendungsbereich

Der Spread Operator (...) ermöglicht es, Elemente eines iterierbaren Objekts (z.B. eines Arrays) in einzelne Elemente zu verteilen oder Objekte zusammenzuführen.

1

2

Beispiel

```
let arr = [...arr1, ...arr2];
```

```
let obj = { ...obj1, ...obj2 };
```

Dies vereinfacht den Umgang mit Arrays und Objekten.

Arrow Functions: Kurze und prägnante Funktionen

Anwendungsbereich

Arrow Functions sind eine Kurzschreibweise zur Deklaration von Funktionen, mit optionaler impliziter Rückgabe und lexikalischer this-Bindung.

Beispiel

```
const add = (a, b) => a + b;
```

Arrow Functions sind besonders nützlich für kurze Callbacks.

Rest Parameters: Eine unbestimmte Anzahl von Argumenten darstellen

1

Anwendungsbereich

Rest Parameters ermöglichen es, eine unbestimmte Anzahl von Argumenten als Array darzustellen.

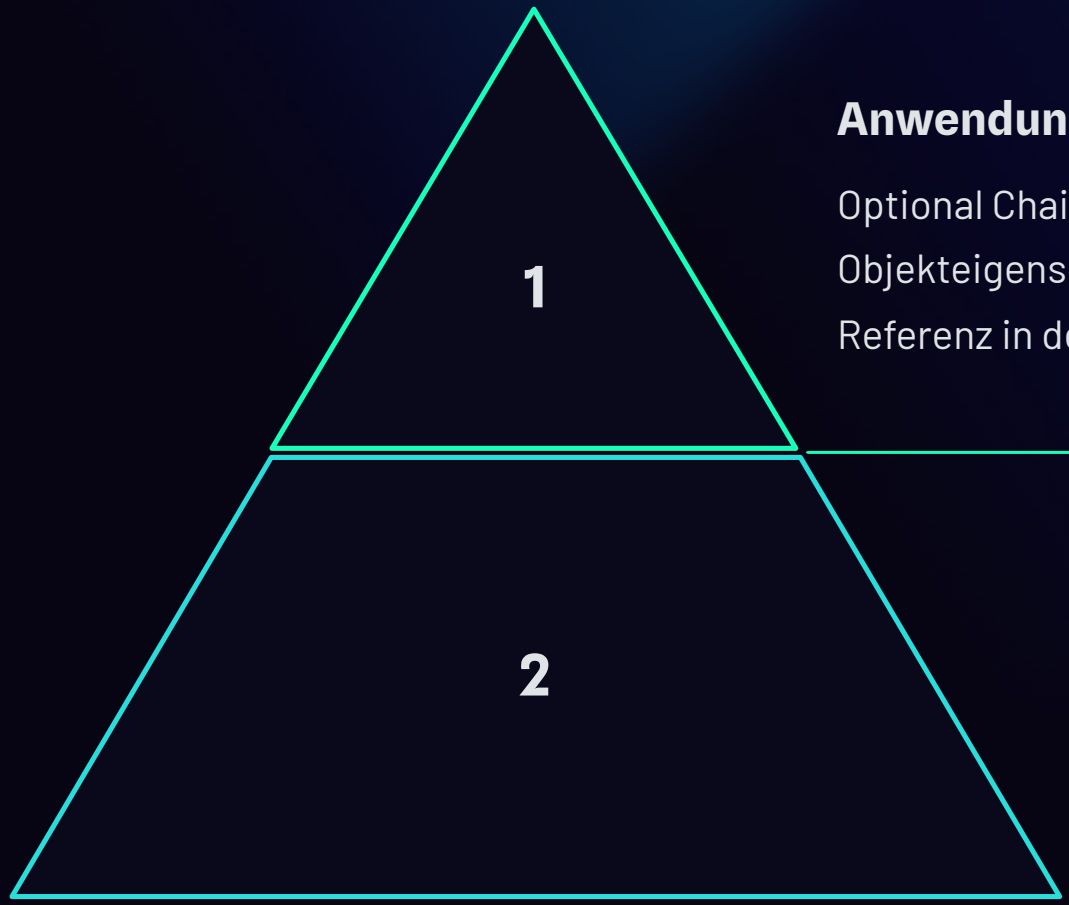
2

Beispiel

```
function sum(...numbers) { /*...*/ }
```

Dies ermöglicht die Erstellung flexibler und wiederverwendbarer Funktionen.

Optional Chaining: Sicheres Zugreifen auf verschachtelte Objekteigenschaften



Anwendungsbereich

Optional Chaining ermöglicht es, auf verschachtelte Objekteigenschaften zuzugreifen, ohne prüfen zu müssen, ob jede Referenz in der Kette null oder undefiniert ist.

Beispiel

```
const value = obj?.prop?.nestedProp;
```

Dies vereinfacht den Code und reduziert Fehler.

Nullish Coalescing Operator: Standardwerte bereitstellen

1

Anwendungsbereich

Der Nullish Coalescing Operator (??) stellt einen Standardwert bereit, wenn eine Variable null oder undefiniert ist.

2

Beispiel

```
const value = input ?? defaultValue;
```

Dies vereinfacht den Code und reduziert Fehler.

for...of Loop: Über iterierbare Objekte iterieren

Anwendungsbereich

Die for...of Schleife iteriert über iterierbare Objekte (wie Arrays, Strings und Generatoren).

Beispiel

```
for (const element of iterable) { /*...*/ }
```

Dies ermöglicht es, direkt mit den Elementen zu arbeiten.

Object.entries(): Objekte als Arrays von Schlüssel-Wert-Paaren darstellen

1

2

Anwendungsbereich

Object.entries() gibt ein Array der aufzählbaren Eigenschaften [key, value] eines Objekts zurück.

Beispiel

```
const entries = Object.entries(obj);
```

Dies ermöglicht eine einfachere Manipulation und Iteration.

Object.values( ): Objekte als Arrays von Werten darstellen



Anwendungsbereich

Object.values() gibt ein Array der aufzählbaren Eigenschaftswerte eines Objekts zurück.



Beispiel

```
const values = Object.values(obj);
```

Dies ermöglicht eine einfachere Manipulation und Iteration.

Array.flat(): Verschachtelte Arrays vereinfachen

Anwendungsbereich

Array.flat() erstellt ein neues Array, in dem alle Unterarray-Elemente rekursiv bis zur angegebenen Tiefe verkettet werden.

Beispiel

```
const flattened = arr.flat(depth);
```

Dies vereinfacht die Array-Manipulation und -Iteration.

Array.flatMap(): Mapping und Flattening in einem Schritt

Anwendungsbereich

Array.flatMap() mappt zuerst jedes Element mit einer Mapping-Funktion und vereinfacht dann das Ergebnis in einem neuen Array.



Beispiel

```
const flatMapped =
arr.flatMap(callback);
```

Dies ermöglicht es, Arrays in einem einzigen Vorgang zu mappen und zu vereinfachen.

Intersection Observer API: Element-Sichtbarkeit erkennen

1

Anwendungsbereich

Die Intersection Observer API ermöglicht es, effizient zu erkennen, wann Elemente sichtbar werden oder sich mit anderen Elementen schneiden.

2

Beispiel

```
const observer = new IntersectionObserver(callback, options);
```

Dies ermöglicht performantes Lazy-Loading, Infinite Scrolling und andere Effekte.

